

Qualification Round 2026

OFFICIAL SOLUTION MANUAL

Problem A — A History of Computing

Below is the chronological sequence of milestone achievements in the history of computer science and technology as outlined in the problem:

- 1936:** Alan Turing publishes the idea of the Turing Machine.
- 1947:** Invention of the transistor at Bell Labs.
- 1958:** Invention of the integrated circuit by Jack Kilby.
- 1969:** Creation of ARPANET, the precursor to the Internet.
- 1981:** Release of the IBM Personal Computer.
- 1991:** Public release of the World Wide Web by Tim Berners-Lee.
- 2000:** Dot-com era / growth of internet technology.
- 2007:** Release of the first Apple iPhone.
- 2012:** Rise of deep learning and AI breakthroughs.
- 2022:** Explosion of generative AI tools such as OpenAI models.

Problem B — Pixel Art

Solution Idea

Checkerboard: A checkerboard pattern alternates dynamically between 0 and 1. If the sum of the coordinate indices (**row + column**) is even, we assign 0; otherwise, we assign 1.

Diamond: The diamond structure is symmetrically centered within the grid. For each cell, we compute its absolute Manhattan distance from the grid center point. If this combined distance is less than or equal to the center index, the cell falls inside the diamond shape and is marked as 1; otherwise, it is assigned 0.

Python Solution

```
def generate_shape(n, shape):
    grid = [[0 for _ in range(n)] for _ in range(n)]

    if shape == "checkerboard":
        for i in range(n):
            for j in range(n):
                grid[i][j] = (i + j) % 2

    elif shape == "diamond":
        center = n // 2

        for i in range(n):
            for j in range(n):
                if abs(i - center) + abs(j - center) <= center:
                    grid[i][j] = 1

    return grid

# Example Usage
n = 5
shape = "diamond"
result = generate_shape(n, shape)

for row in result:
    print(*row)
```

Time Complexity: $O(n^2)$

Problem C — Moore's Law

Given the base exponential growth function according to Moore's Law:

$$T(t) = T_0 \times 2^{t/2}$$

Where the Intel 4004 transistor count in 1971 is defined as $T_0 = 2300$, and the timespan from 1971 to 2026 is $t = 55$ years.

(a) Prediction for 2026

Substituting the variables into the formula:

$$T(55) = 2300 \times 2^{55/2} = 2300 \times 2^{27.5} \approx 2300 \times 1.9 \times 10^8$$

$$T(55) \approx 4.4 \times 10^{11} \text{ (Approximately 440 billion transistors)}$$

(b) Comparison

Modern flagship microprocessors (such as the Apple M5 series) are estimated to scale around tens to hundreds of billions of transistors. This emphasizes that Moore's Law remained remarkably precise for over five decades, though contemporary physical, thermal, and sub-nanometer manufacturing limitations have slowly started to decelerate this exponential phase.

(c) Actual Doubling Period

To evaluate the actual historical doubling period p , we use:

$$T = 2300 \times 2^{55/p}$$

Assuming modern high-density integration processors contain roughly 10^{11} transistors:

$$10^{11} = 2300 \times 2^{55/p}$$

Solving for the exponent yields an approximate real doubling period of $p \approx 2.3$ years, demonstrating a slightly slower pace than Moore's initial 2-year estimate.

Problem D — The Simultaneous Strike

(a) Logical Flaw Analysis

The core bug stems from verifying a Knockout (KO) event immediately following the processing of each atomic simulation event. If two attacks occur simultaneously on the exact same frame, the evaluation loop might terminate prematurely if the first processed attack reduces a player's HP to 0. This directly violates the definitive rule: *"All events on the same frame must be applied before checking for KO."* Consequently, simultaneous multi-hit knockouts are handled erroneously.

Correct Python Solution

```
def processGame(events, H):
    hp1 = H
    hp2 = H

    # Sort by frame
    events.sort(key=lambda x: x[1])

    i = 0
    while i < len(events):
        current_frame = events[i][1]

        # Process all events in the same frame
        while i < len(events) and events[i][1] == current_frame:
            player, frame, attack = events[i]

            if player == 1:
                hp2 -= attack
            else:
                hp1 -= attack
            i += 1

        # Clamp HP to avoid negative values
        hp1 = max(0, hp1)
        hp2 = max(0, hp2)

        # Check KO only after entire frame processing is complete
        if hp1 == 0 or hp2 == 0:
            break

    return [hp1, hp2]

# Example Implementation
events = [
    (1, 512, 20),
    (2, 512, 20)
]
print(processGame(events, 15)) # Output: [0, 0]
```

تعقيد الوقت هو $O(n \log n)$ بسبب الفرز.

Problem E — PageRank in the Age of AI Slop

(a) How PageRank Works

PageRank models internet browsing via a "random surfer" paradigm. With a damping probability d , the surfer navigates along existing hyperlinks; conversely, with a probability $1 - d$, the surfer teleports uniformly to any random page across the web. The algorithmic framework is inherently recursive because a target page's structural importance relies strictly on the pre-existing importance weights of all inbound source pages linking to it.

(b) Total AI PageRank

Let N represent total pages, a be AI-generated pages, and h be human-curated pages. Let the ratio of AI content be defined as $\alpha = a/N$. Because closed AI link farms only link internally to their own pages, they successfully trap PageRank. The total PageRank accumulated within the AI network is mathematically derived as:

$$S_A = \alpha / (1 - d + d\alpha)$$

(c) Tipping Point Analysis

The tipping point occurs when AI-generated pages capture exactly half of the total global PageRank authority:

$$S_A = S_H = 1/2$$

Substituting this equilibrium into the derived structural equation:

$$\alpha / (1 - d + d\alpha) = 1/2$$

Solving explicitly for critical density α^* :

$$\alpha^* = (1 - d) / (2 - d)$$

For standard damping factor constants where $d = 0.85$:

$$\alpha^* \approx 0.13$$

This means that AI-generated synthetic pages need to constitute **only about 13%** of the total web corpus to control over 50% of the entire Internet's PageRank authority. This exposes an existential weakness in hyperlink-based ranking models: dense link-farms and closed, automated generative loops completely game and skew systemic relevance metrics without producing high-quality information.